Worked for 3m 58s ›

তোমার জন্য Final Singular Research Path

তুমি আর একাধিক route নিয়ে ভাববে না। পুরো চার মাস নিচের **একটি research path** অনুসরণ করবে:

Final Title

DynaCampus-FR: Domain-Adaptive, Quality-Aware Open-Set Face Recognition with Training-Free Dynamic Enrollment for Low-Quality Campus CCTV

সহজভাবে project-টি হবে:

> Low-quality campus CCTV video থেকে known student/faculty চিনবে, unknown person reject করবে, video-র একাধিক frame ব্যবহার করে stable decision দেবে এবং নতুন student/faculty যোগ করার সময় পুরো model retrain করতে হবে না।

তোমাদের বর্তমান report-এ 7-person LFW dataset, 40×40 image, Haar Cascade, ছোট CNN, softmax confidence, desktop app এবং dashboard ব্যবহার করা হয়েছে। এগুলো basic prototype হিসেবে রাখা যাবে, কিন্তু final research contribution হিসেবে নয়।

FYDP_II_Report__CCTV_Based_Face... FYDP_II_Report__CCTV_Based_Face...

---

1. এক লাইনে তোমাদের নতুন Research

তোমরা একটি pretrained face-recognition backbone ব্যবহার করবে। নতুন student এলে তার কয়েকটি ছবি থেকে **embedding template** তৈরি করে gallery-তে যোগ করবে। এরপর CCTV video থেকে face detect, track, quality measure এবং multiple-frame embedding combine করে open-set recognition করা হবে।

পুরো system:

```
Enrollment Photo/Video
        ↓
Face Detection: SCRFD
        ↓
Face Alignment
        ↓
Recognition Backbone: AdaFace
        ↓
CCTV Domain Adapter
        ↓
Quality-Aware Identity Template
        ↓
FAISS Gallery
```

CCTV side:

```
CCTV Video
    ↓
Frame Extraction
    ↓
SCRFD Face Detection
    ↓
ByteTrack Tracking
    ↓
Face Alignment
    ↓
AdaFace Embedding
    ↓
CCTV Domain Adapter
    ↓
Quality-Aware Multi-Frame Aggregation
    ↓
Open-Set Matching
    ↓
KNOWN / UNKNOWN / AMBIGUOUS / LOW QUALITY
```

---

2. এই Path কেন নির্বাচন করছি

এই path-এ একই research problem-এর মধ্যে teacher-এর প্রায় সব suggestion সমাধান হচ্ছে:

Teacher-এর concern	**তোমাদের final solution**
Data augmentation নেই	CCTV-specific augmentation থাকবে
Proper dataset নেই	QMUL-SurvFace + নিজেদের CampusFace-BD
নতুন student এলে retraining লাগে	Training-free dynamic enrollment
Desktop/mobile app meaningful নয়	App বাদ, research pipeline হবে মূল কাজ
ViT ব্যবহার করা যায় কি না	FaceLiVTv2 দিয়ে comparison
RAG কোথায়	Core research থেকে বাদ, future work
Models outdated	SCRFD, AdaFace, FaceLiVTv2, ByteTrack
Low novelty	Domain Adapter + augmented enrollment + temporal aggregation
Unknown detection weak	Calibrated open-set recognition

Teacher-এর concern	তোমাদের final solution
Single-frame unstable	Multi-frame quality-aware recognition

এই direction-এর **publishable potential আছে**, কিন্তু publication guarantee করা যায় না। Publishability নির্ভর করবে identity-disjoint evaluation, clean experiments, measurable improvement, ethical data collection এবং paper writing-এর ওপর।

---

3. RAG নিয়ে Final Decision

RAG implement করবে না

RAG face detection বা face recognition improve করে না। এটি textual documents বা records retrieve করে LLM দিয়ে answer তৈরির জন্য ব্যবহার হয়।

তোমাদের সময় এবং research focus বিবেচনায়:

```
RAG = Future Work
Mobile App = বাদ
Desktop App = বাদ
Large Dashboard = বাদ
Anti-spoofing = Future Work
```

শেষে demonstration-এর জন্য একটি খুব ছোট `Streamlit` interface বানাতে পারো। কিন্তু সেটি research contribution হবে না।

---

4. Vision Transformer কোথায় থাকবে

Vision Transformer তোমাদের primary model হবে না। কারণ শুরুতেই ViT integration করলে project unnecessarily কঠিন হয়ে যাবে।

তোমরা তিনটি recognizer compare করবে:

1. **ArcFace/InsightFace** — baseline
2. **AdaFace** — primary research backbone
3. **FaceLiVTv2-S** — latest lightweight hybrid CNN-Transformer comparison

AdaFace low-quality samples বিবেচনা করে quality-adaptive margin ব্যবহার করে এবং low-quality face benchmarks-এ evaluate করা হয়েছে। GitHub +1

FaceLiVTv2 একটি 2026 hybrid CNN-Transformer model; official repository এবং pretrained weights আছে। তবে এটি বর্তমানে preprint/under-review work, তাই paper-এ সেই status

পরিষ্কারভাবে লিখতে হবে। GitHub +2

Final method প্রথমে AdaFace-এ তৈরি করবে। পরে একই dataset-এ FaceLiVTv2-S দিয়ে comparison করবে।

---

5. প্রথমে Basic Terms বোঝো

5.1 Face Detection

ছবিতে মুখ কোথায় আছে সেটি বের করা।

Input:

```
Full CCTV frame
```

Output:

```
Face bounding box
Facial landmarks
Detection confidence
```

তোমরা ব্যবহার করবে:

```
SCRFD
```

SCRFD accuracy এবং computation-এর balance মাথায় রেখে তৈরি করা face detector এবং InsightFace project-এ implementation আছে। GitHub +1

5.2 Face Alignment

কেউ একটু বাঁকা হয়ে তাকালে তার চোখ, নাক ও মুখের landmarks ব্যবহার করে face crop-টিকে সোজা করা।

```
Unaligned face → 112 × 112 aligned face
```

সব recognizer একই ধরনের aligned input পাবে।

5.3 Face Embedding

Model একটি face image-কে একটি numerical vector-এ convert করে।

উদাহরণ:

```
Face Image → [0.03, -0.17, 0.42, ..., 0.08]
```

সাধারণত embedding হবে:

```
512-dimensional vector
```

এটিকে face-এর numerical signature হিসেবে ভাবতে পারো।

5.4 Gallery

যাদের system চেনে, তাদের identity template-এর collection হলো gallery।

```
S001 → embedding template
S002 → embedding template
S003 → embedding template
```

5.5 Probe

CCTV থেকে আসা যে face-টির পরিচয় বের করতে হবে, সেটি probe।

5.6 Cosine Similarity

দুটি embedding কতটা similar তা measure করে:

$$\text{cosine}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

Score বেশি হলে faces similar হওয়ার সম্ভাবনা বেশি।

5.7 Open-Set Recognition

System-এর gallery-তে না থাকা person-কে জোর করে কোনো known identity দেওয়া যাবে না।

System-এর possible decision হবে:

```
KNOWN
UNKNOWN
```

```
AMBIGUOUS
INSUFFICIENT_QUALITY
```

5.8 Dynamic Enrollment

নতুন student এলে model retrain না করে:

```
New Student Images
      ↓
Embedding Extraction
      ↓
Template Creation
      ↓
Gallery Insert
```

এটাই semester-wise retraining problem সমাধান করবে।

---

6. তোমাদের Exact Research Contributions

Final paper-এ পাঁচটি contribution থাকবে।

Contribution 1: CampusFace-BD Dataset

নিজেদের consent-based Bangladeshi campus CCTV dataset।

Contribution 2: CCTV Domain Adapter

Pretrained backbone freeze থাকবে। শুধু ছোট একটি adapter train করা হবে, যা clean enrollment embedding এবং degraded CCTV embedding-এর gap কমাবে।

Contribution 3: CCTV-Augmented Dynamic Enrollment

একটি clean enrollment photo থেকে কয়েকটি realistic CCTV-style variant তৈরি করে robust identity template বানানো হবে।

Contribution 4: Quality-Aware Multi-Frame Aggregation

একটি blurry frame-এর ওপর decision না দিয়ে track-এর multiple good frames combine করা হবে।

Contribution 5: Open-Set Four-State Decision

System শুধু known/unknown নয়, নিচের চারটি state আলাদা করবে:

```
KNOWN
UNKNOWN
```

```
AMBIGUOUS
INSUFFICIENT_QUALITY
```

---

7. Final Research Questions

Paper-এ এই RQগুলো ব্যবহার করবে।

RQ1

How much do low resolution, motion blur, low illumination, compression and occlusion affect modern face-recognition models in campus CCTV conditions?

RQ2

Can a lightweight CCTV Domain Adapter reduce the gap between clean enrollment images and degraded CCTV probes?

RQ3

Does CCTV-aware augmented enrollment improve one-shot dynamic enrollment without retraining the recognition backbone?

RQ4

Does quality-aware multi-frame aggregation outperform single-frame recognition?

RQ5

How does the primary CNN-based AdaFace model compare with the hybrid Transformer-based FaceLiVTv2 under the same CCTV conditions?

RQ6

Can new identities be added without retraining while maintaining reliable open-set recognition?

---

8. কোন Platforms কোথায় ব্যবহার করবে

Complete Platform Plan

কাজ	Platform
Code লেখা	VS Code
Version control	Git + private GitHub repository

কাজ	Platform
Beginner experiments	Google Colab
Final scripts	VS Code + Python
GPU experiments	Google Colab GPU
Dataset storage	Restricted Google Drive + external backup
Video annotation	Self-hosted CVAT
Paper writing	Overleaf
References	Zotero
Diagrams	diagrams.net
Experiment records	CSV + TensorBoard
Final demo	Streamlit
Team task tracking	GitHub Projects

CVAT image ও video annotation এবং Track Mode support করে। Video sequence-এ এক person-এর face track annotate করার জন্য এটি ব্যবহার করা যাবে। CVAT D... +1

---

9. Complete Tech Stack

Core Language

```
Python 3.10
```

Python 3.10 নির্বাচন করো কারণ computer-vision repositories এবং pretrained model code-এর compatibility সাধারণত তুলনামূলকভাবে stable।

Core Libraries

```
PyTorch
Torchvision
OpenCV
InsightFace
ONNX Runtime
NumPy
```

```
Pandas
Scikit-learn
SciPy
Matplotlib
Albumentations
FAISS
PyYAML
TQDM
TensorBoard
Streamlit
```

Albumentations দিয়ে blur, downscale, compression এবং অন্যান্য CCTV-like augmentation করা যাবে। এর official transforms-এর মধ্যে downscale ও compression simulation রয়েছে। Albumentations +1

FAISS dense embedding collection-এর মধ্যে similarity search করার জন্য তৈরি। তোমাদের student gallery search-এ এটি ব্যবহার হবে। GitHub +1

Models

```
Detector: SCRFD
Baseline Recognizer: InsightFace ArcFace
Primary Recognizer: AdaFace IR-50
Transformer Comparison: FaceLiVTv2-S
Tracker: ByteTrack
```

ByteTrack detection boxes-কে video frames-এর মধ্যে associate করে persistent track তৈরি করে। GitHub +1

---

10. Hardware Plan

Development Computer

Recommended:

```
Windows 10/11
RAM: 8 GB minimum, 16 GB preferred
Storage: At least 100 GB free
GPU: Not mandatory locally
```

Training and Experiment GPU

Use:

```
Google Colab GPU
```

Code কখনো শুধু Colab notebook-এ রেখে দেবে না। Code GitHub repository-তে থাকবে; Colab repository clone করে run করবে।

Camera

Dataset collection-এর জন্য:

```
2 fixed cameras
1 smartphone for enrollment photos
Tripod or high fixed mount
```

Camera setup:

```
CAM_A: frontal corridor view
CAM_B: side/diagonal view
Camera height: approximately 2.3–2.8 meters
```

Actual campus CCTV ব্যবহার করতে হলে institution-এর written permission নিতে হবে। Permission না পাওয়া পর্যন্ত কোনো regular campus surveillance footage ব্যবহার করবে না।

---

11. Account এবং Software Setup

প্রথমে নিচের accounts/software তৈরি বা install করো:

1. GitHub account
2. Google account
3. Google Colab
4. Google Drive research folder
5. Overleaf account
6. Zotero
7. VS Code
8. Git
9. Miniconda
10. Docker Desktop—CVAT local চালানোর জন্য

---

12. Main Folder Structure

GitHub repository-এর নাম দাও:

```
DynaCampus-FR
```

Repository structure:

```
DynaCampus-FR/
│
├── README.md
├── .gitignore
├── environment.yml
├── requirements.txt
│
├── configs/
│   ├── paths.yaml
│   ├── models.yaml
│   ├── collection.yaml
│   ├── augmentation.yaml
│   ├── adapter.yaml
│   └── evaluation.yaml
│
├── notebooks/
│   ├── 00_environment_check.ipynb
│   ├── 01_face_detection_demo.ipynb
│   ├── 02_face_alignment_demo.ipynb
│   ├── 03_embedding_similarity_demo.ipynb
│   ├── 04_dynamic_enrollment_demo.ipynb
│   └── 05_open_set_demo.ipynb
│
├── src/
│   ├── detection/
│   │   └── scrfd_detector.py
│   │
│   ├── alignment/
│   │   └── face_aligner.py
│   │
│   ├── recognition/
│   │   ├── insightface_recognizer.py
│   │   ├── adaface_recognizer.py
│   │   └── facelivt_recognizer.py
│   │
│   ├── augmentation/
│   │   └── cctv_augmentations.py
│   │
│   ├── tracking/
│   │   └── bytetrack_tracker.py
│   │
│   ├── quality/
│   │   └── face_quality.py
│   │
│   ├── adapter/
│   │   ├── residual_adapter.py
│   │   ├── triplet_dataset.py
│   │   └── losses.py
│   │
│   ├── gallery/
│   │   ├── build_gallery.py
```

```
│   │   └── faiss_gallery.py
│   │
│   ├── open_set/
│   │   ├── matcher.py
│   │   └── threshold_calibration.py
│   │
│   ├── aggregation/
│   │   └── temporal_aggregation.py
│   │
│   ├── evaluation/
│   │   ├── verification_metrics.py
│   │   ├── open_set_metrics.py
│   │   └── runtime_metrics.py
│   │
│   └── utils/
│       ├── config.py
│       ├── io.py
│       └── seed.py
│
├── scripts/
│   ├── extract_frames.py
│   ├── detect_faces.py
│   ├── extract_embeddings.py
│   ├── prepare_adapter_data.py
│   ├── train_adapter.py
│   ├── build_gallery.py
│   ├── calibrate_thresholds.py
│   ├── evaluate_open_set.py
│   ├── evaluate_augmentations.py
│   ├── evaluate_aggregation.py
│   └── benchmark_runtime.py
│
├── results/
│   ├── raw/
│   ├── tables/
│   ├── figures/
│   ├── logs/
│   └── checkpoints/
│
├── tests/
│
└── docs/
    ├── research_protocol/
    ├── consent/
    ├── dataset_card/
    ├── experiment_log/
    └── paper_notes/
```

---

13. Dataset GitHub-এর ভেতরে থাকবে না

Face data sensitive biometric data। তাই repository-তে শুধু code থাকবে।

Separate restricted storage:

```
CampusFaceBD_Data/
│
├── private/
│   ├── identity_map_encrypted.csv
│   └── consent_forms/
│
├── raw/
│   ├── enrollment/
│   ├── cctv_session_1/
│   └── cctv_session_2/
│
├── processed/
│   ├── frames/
│   ├── detected_faces/
│   ├── aligned_faces/
│   ├── tracks/
│   └── embeddings/
│
├── metadata/
│   ├── participants.csv
│   ├── videos.csv
│   ├── frames.csv
│   ├── tracks.csv
│   └── quality.csv
│
└── splits/
    ├── split_01.json
    ├── split_02.json
    └── split_03.json
```

`.gitignore`:

gitignore

```
data/
CampusFaceBD_Data/
*.mp4
*.avi
*.mov
*.jpg
*.jpeg
*.png
*.npy
*.npz
*.onnx
*.pt
```

```
*.pth
__pycache__/
.env
```

Pretrained model licensing-ও check করবে। InsightFace Python code permissive হলেও official pretrained model packages academic/non-commercial research restriction বহন করতে পারে। PyPI +1

---

14. Environment Setup — একদম শুরু থেকে

Step 1: Repository তৈরি

GitHub-এ private repository তৈরি করো:

```
DynaCampus-FR
```

তারপর VS Code terminal:

Bash

```
git clone YOUR_PRIVATE_REPOSITORY
cd DynaCampus-FR
```

Step 2: Conda Environment

Bash

```
conda create -n dynacampus python=3.10 -y
conda activate dynacampus
python -m pip install --upgrade pip
```

Step 3: Basic Packages

Bash

```
pip install numpy pandas scipy scikit-learn matplotlib pillow pyyaml tqdm
pip install opencv-python
pip install albumentations
pip install faiss-cpu
pip install tensorboard streamlit
```

Step 4: InsightFace Baseline

CPU machine হলে:

```bash
pip install insightface onnxruntime
```

Colab/GPU হলে:

```bash
pip install insightface onnxruntime-gpu
```

Step 5: PyTorch

PyTorch installation command CUDA এবং computer configuration অনুযায়ী বদলায়। Official PyTorch installer থেকে Windows/Linux, pip এবং correct CUDA option select করে command নাও। PyTorch +1

Colab-এ সাধারণত PyTorch preinstalled থাকবে। Verify:

```python
import torch

print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

Step 6: Environment Lock

সবকিছু সফলভাবে install হওয়ার পরে:

```bash
pip freeze > requirements-lock.txt
```

এটি তোমাদের exact working environment preserve করবে।

15. প্রথম Working Face Detection

`notebooks/01_face_detection_demo.ipynb` তৈরি করো।

Python

Run

```python
import cv2
from insightface.app import FaceAnalysis

providers = [
    "CUDAExecutionProvider",
    "CPUExecutionProvider",
]

app = FaceAnalysis(
    name="buffalo_l",
    providers=providers,
)

app.prepare(
    ctx_id=0,
    det_size=(640, 640),
)

image = cv2.imread("test_image.jpg")

if image is None:
    raise FileNotFoundError("test_image.jpg পাওয়া যায়নি")

faces = app.get(image)

print("Detected faces:", len(faces))

for index, face in enumerate(faces):
    x1, y1, x2, y2 = face.bbox.astype(int)

    cv2.rectangle(
        image,
        (x1, y1),
        (x2, y2),
        (0, 255, 0),
        2,
    )

    cv2.putText(
        image,
        f"Face {index + 1}",
        (x1, max(20, y1 - 10)),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 0),
```

```
        2,
    )

cv2.imwrite("detected_output.jpg", image)
```

Expected result:

```
Input image → face-এর চারপাশে bounding box
```

প্রথম milestone:

> নিজের ছবি এবং group image-এ face detection successful।

---

16. প্রথম Face Embedding এবং Similarity Test

`notebooks/03_embedding_similarity_demo.ipynb`:

Python Run

```python
import numpy as np
import cv2
from insightface.app import FaceAnalysis

app = FaceAnalysis(
    name="buffalo_l",
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"],
)

app.prepare(ctx_id=0, det_size=(640, 640))


def get_embedding(image_path: str) -> np.ndarray:
    image = cv2.imread(image_path)

    if image is None:
        raise FileNotFoundError(image_path)

    faces = app.get(image)

    if len(faces) != 1:
        raise ValueError(
            f"{image_path}: exactly one face expected, found {len(faces)}"
        )

    embedding = faces[0].normed_embedding.astype(np.float32)
    return embedding
```

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    a = a / np.linalg.norm(a)
    b = b / np.linalg.norm(b)
    return float(np.dot(a, b))


person_a_1 = get_embedding("person_a_1.jpg")
person_a_2 = get_embedding("person_a_2.jpg")
person_b_1 = get_embedding("person_b_1.jpg")

same_score = cosine_similarity(person_a_1, person_a_2)
different_score = cosine_similarity(person_a_1, person_b_1)

print("Same person score:", same_score)
print("Different person score:", different_score)
```

এখানে কোনো classifier train করা হচ্ছে না।

তুমি observe করবে:

```
Same person similarity সাধারণত বেশি
Different person similarity সাধারণত কম
```

কিন্তু কোনো fixed threshold এখনই নির্ধারণ করবে না।

---

17. প্রথম Dynamic Enrollment Demo

Folder:

```
demo_enrollment/
├── S001/
│   ├── image_01.jpg
│   ├── image_02.jpg
│   └── image_03.jpg
│
├── S002/
│   ├── image_01.jpg
│   ├── image_02.jpg
│   └── image_03.jpg
```

প্রতিটি identity-এর embeddings average করবে:

$$p_i = \text{normalize}\left(\frac{1}{N}\sum_{k=1}^{N} z_k\right)$$

এখানে:

```
z_k = একটি image-এর embedding
p_i = person i-এর identity prototype
```

নতুন student যোগ করতে:

```bash
python scripts/build_gallery.py \
  --subject-id S003 \
  --input-folder demo_enrollment/S003
```

Expected output:

```
Identity S003 inserted into gallery.
Model retraining: NO
```

এই feature-টি প্রথম দুই সপ্তাহের মধ্যে working করতে হবে।

---

18. Proper Dataset Plan

তোমাদের research-এ দুই ধরনের dataset থাকবে। এটি দুইটি route নয়; একই research-এর দুইটি evaluation source।

Dataset A: QMUL-SurvFace

Purpose:

```
Public surveillance benchmark
Open-set evaluation
Low-resolution face recognition
```

QMUL-SurvFace native surveillance faces এবং open-set evaluation protocol-এর জন্য তৈরি। Official dataset, protocols এবং test code প্রকাশিত আছে। QMUL-SurvFace +1

এটি download করে রাখবে:

```
CampusFaceBD_Data/public/QMUL-SurvFace/
```

Random Kaggle copy ব্যবহার না করে official source এবং license অনুসরণ করবে।

Dataset B: CampusFace-BD

Purpose:

```
Bangladeshi campus context
Dynamic enrollment
Multiple sessions
Multiple cameras
Known and unknown identities
CCTV degradation analysis
```

Target

```
Participants: 60
Sessions: 3
Cameras: 2
Enrollment images: 15 per participant
Enrollment video: 15 seconds per participant
CCTV sessions: 2 separate days
```

---

19. Ethics এবং Permission — Data Collection-এর আগে

কোনো face data collect করার আগে:

1. Supervisor approval
2. Department/institution permission
3. Written participant consent
4. Data access policy
5. Withdrawal procedure
6. Retention/deletion date
7. Publication permission আলাদা করে লেখা
8. Anonymized subject IDs
9. Restricted storage
10. Non-consenting bystander recording নিষিদ্ধ

Consent form-এ পরিষ্কার থাকবে:

```
Why data is collected
What camera conditions are used
Who can access it
How long it will be stored
Whether sample images may appear in paper
How participant can withdraw
Whether dataset will remain private or be released
```

Public release permission না থাকলে dataset public করবে না। তবুও paper-এ dataset protocol, statistics এবং anonymized examples publish করা যায়।

---

20. Participant ID System

Real name dataset file-এ ব্যবহার করবে না।

```
S001
S002
S003
...
S060
```

Private mapping:

```csv
subject_id,real_name,consent_date,withdrawn
S001,Private Name,2026-07-20,no
```

এই file encrypted private folder-এ থাকবে।

Public research metadata:

```csv
subject_id,age_group,session_count,has_glasses
S001,20-29,3,yes
```

Demographic fields collect করলে ethics approval এবং clear research justification থাকতে হবে।

---

21. Data Collection Protocol

Session E — Enrollment Session

Environment:

```
Controlled indoor room
Normal illumination
Plain background
Smartphone camera
Distance: approximately 1 meter
```

প্রতিটি participant-এর:

```
5 frontal photos
3 slight-left photos
3 slight-right photos
```

```
2 looking slightly upward
2 looking slightly downward
15-second slow head-movement video
```

Naming:

```
S001_E_IMG_001.jpg
S001_E_IMG_002.jpg
S001_E_VIDEO_001.mp4
```

Session C1 — CCTV Session 1

Environment:

```
Normal daylight/indoor light
Two fixed cameras
Corridor-style setup
```

প্রতিটি participant করবে:

```
Walk toward camera
Walk away from camera
Walk left-to-right
Walk right-to-left
Pause at near distance
Pause at medium distance
Pause at far distance
```

Suggested distances:

```
Near: 2 meters
Medium: 5 meters
Far: 8 meters
```

Naming:

```
S001_C1_CAMA_001.mp4
S001_C1_CAMB_001.mp4
```

Session C2 — CCTV Session 2

অন্য দিন record করবে।

Conditions:

```
Lower illumination
Different clothes
Optional glasses
Partial occlusion
Slightly faster walking
Small group scene
```

Naming:

```
S001_C2_CAMA_001.mp4
S001_C2_CAMB_001.mp4
```

---

22. Video Frame Sampling

সব 25 বা 30 FPS frame রাখতে হবে না।

Use:

```
5 FPS sampling
```

Example:

Bash

```bash
python scripts/extract_frames.py \
  --input S001_C1_CAMA_001.mp4 \
  --output processed/frames/S001/C1/CAMA \
  --fps 5
```

Expected dataset scale:

```
60 identities
2 CCTV sessions
2 cameras
Approximately 30 seconds per video
5 sampled frames per second
```

এতে manageable কিন্তু meaningful পরিমাণ frame পাওয়া যাবে।

---

23. Metadata Structure

`videos.csv`:

```csv
video_id,subject_id,session_id,camera_id,lighting,occlusion,path
V0001,S001,C1,CAMA,normal,none,raw/cctv_session_1/S001_C1_CAMA_001.mp4
```

`frames.csv`:

```csv
frame_id,video_id,timestamp,path
F000001,V0001,0.0,processed/frames/F000001.jpg
```

`tracks.csv`:

```csv
track_id,subject_id,video_id,start_frame,end_frame,is_verified
T000001,S001,V0001,1,140,yes
```

`faces.csv`:

```csv
face_id,track_id,frame_id,x1,y1,x2,y2,det_conf,face_width,path
FACE000001,T000001,F000001,100,50,190,150,0.96,90,processed/faces/FACE000001.jpg
```

`quality.csv`:

```csv
face_id,blur_score,size_score,pose_score,model_quality,final_quality
FACE000001,0.78,0.80,0.90,0.83,0.82
```

---

24. Annotation Plan

সব frame manually bounding-box annotate করবে না।

Automatic part

SCRFD ব্যবহার করে:

```
Face detection
Bounding box
```

```
Landmarks
Face crop
```

Manual verification

Team verify করবে:

```
Correct subject?
Wrong detection?
Multiple faces?
Track broken?
Face usable?
```

CVAT ব্যবহার

শুধু প্রায়:

```
1,000 representative frames
```

manually annotate করবে detector evaluation-এর জন্য।

CVAT Track Mode sequence-এর মধ্যে moving object track edit এবং merge করতে পারে।

CVAT Docume... +1

---

25. Dataset Split — Data Leakage বন্ধ করার জন্য

একই video-এর adjacent frames randomভাবে train/test split করবে না।

তিনটি Identity-Disjoint Split

প্রতিটি split-এ:

```
Development identities: 30
Known test identities: 20
Unknown test identities: 10
```

Development Identities

ব্যবহার হবে:

```
Domain Adapter training
Threshold calibration
Hyperparameter selection
```

Known Test Identities

Enrollment থাকবে, CCTV probes থাকবে।

Unknown Test Identities

Gallery-তে থাকবে না। শুধু CCTV probes থাকবে।

Example Split 1

```
DEV:
S001–S030

TEST KNOWN:
S031–S050

TEST UNKNOWN:
S051–S060
```

Split 2 এবং Split 3-এ groups rotate করবে।

Final result:

```
Mean performance across 3 splits
Standard deviation across 3 splits
```

---

26. Data Augmentation Plan

তোমাদের augmentation random beauty filter হবে না। CCTV degradation simulate করবে।

Basic Augmentations

```
HorizontalFlip
Small rotation
BrightnessContrast
Gamma adjustment
Small crop
```

CCTV Augmentations

```
Downscale
MotionBlur
GaussianBlur
DefocusBlur
JPEGCompression
```

```
ImageCompression
GaussNoise
LowLight
Shadow
ColorShift
Occlusion
```

Severity Levels

```
Severity 0: Clean
Severity 1: Mild
Severity 2: Moderate
Severity 3: Severe
```

Combined Degradations

```
Low resolution + Motion blur
Low light + Noise
Compression + Downscale
Pose + Occlusion
Low light + Motion blur + Compression
```

Albumentations downscale, compression এবং blur-style transformations implement করার জন্য উপযুক্ত। Albumentations +1

---

27. Augmented Dynamic Enrollment

একটি নতুন student-এর clean image থেকে variants তৈরি করবে:

```
Original
Low resolution
Motion blurred
Compressed
Low light
Low resolution + compressed
Low light + noise
Mild occlusion
```

প্রতিটি variant থেকে embedding:

$$z_1, z_2, \ldots, z_N$$

তারপর quality-weighted prototype:

$$p_i = \text{normalize}\left(\frac{\sum_{k=1}^{N} q_k z_k}{\sum_{k=1}^{N} q_k}\right)$$

এখানে:

```
q_k = image quality score
z_k = embedding
p_i = final identity template
```

Compare করবে:

```
E1: One original image
E2: Five original images
E3: One image + CCTV augmentations
E4: Enrollment video
E5: Proposed quality-weighted augmented template
```

---

28. CCTV Domain Adapter

এটিই তোমাদের মূল learnable research component।

Problem

Clean enrollment image embedding:

```
z_clean
```

Low-quality CCTV face embedding:

```
z_cctv
```

একই person হলেও degradation-এর কারণে দুই embedding দূরে চলে যেতে পারে।

Solution

AdaFace backbone freeze করবে:

```
Backbone weights = frozen
```

তার পরে একটি ছোট residual MLP adapter:

```
512 → 256 → 512
```

Equation:

$$z' = \text{normalize}\left(z + g_\theta(z)\right)$$

এখানে:

```
z = pretrained backbone embedding
gθ = trainable domain adapter
z' = adapted embedding
```

শুধু `gθ` train হবে।

Adapter Structure

Python

Run

```python
import torch
from torch import nn
import torch.nn.functional as F


class ResidualAdapter(nn.Module):
    def __init__(
        self,
        embedding_dim: int = 512,
        hidden_dim: int = 256,
        dropout: float = 0.1,
    ) -> None:
        super().__init__()

        self.adapter = nn.Sequential(
            nn.Linear(embedding_dim, hidden_dim),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, embedding_dim),
        )

    def forward(self, embedding: torch.Tensor) -> torch.Tensor:
        adapted = embedding + self.adapter(embedding)
        return F.normalize(adapted, p=2, dim=1)
```

---

29. Adapter Training Data

Development identities থেকে triplet বানাবে:

```
Anchor: clean enrollment face
Positive: same person-এর CCTV/degraded face
```

```
Negative: different person-এর face
```

Example:

```
Anchor   = S003 clean enrollment
Positive = S003 low-quality CCTV
Negative = S017 CCTV
```

Triplet loss:

$$L_{\text{triplet}} = \max\left(0, d(a,p) - d(a,n) + m\right)$$

এখানে:

```
a = anchor
p = positive
n = negative
m = margin
```

Goal:

```
Same identity embeddings closer
Different identity embeddings farther
```

Initial settings:

```
Embedding dimension: 512
Hidden dimension: 256
Margin: 0.2
Batch size: 64
Optimizer: AdamW
Learning rate: 0.0001
Epochs: maximum 30
Early stopping patience: 5
```

Final values development validation data দিয়ে নির্বাচন করবে।

---

30. Quality Score

প্রথমে একটি interpretable quality score তৈরি করবে।

Components:

```
Detection confidence
Face size
Blur score
```

```
Pose score
Embedding quality
```

Formula:

$$q = 0.20q_{\text{det}} + 0.20q_{\text{size}} + 0.20q_{\text{blur}} + 0.15q_{\text{pose}} + 0.25q_{\text{model}}$$

সব score আগে `[0,1]` range-এ normalize করবে।

Blur Score

Variance of Laplacian ব্যবহার করে initial blur estimate:

Python Run

```python
import cv2
import numpy as np


def laplacian_blur_value(face_image: np.ndarray) -> float:
    gray = cv2.cvtColor(face_image, cv2.COLOR_BGR2GRAY)
    return float(cv2.Laplacian(gray, cv2.CV_64F).var())
```

Hardcoded universal threshold ব্যবহার করবে না। Development data দিয়ে normalization এবং threshold ঠিক করবে।

---

31. Tracking এবং Multi-Frame Recognition

একজন person video-তে কয়েক সেকেন্ড দেখা গেলে তার অনেক face frame থাকবে।

ByteTrack দিয়ে:

```
Frame 1: Face → Track T001
Frame 2: Face → Track T001
Frame 3: Face → Track T001
...
```

Track-এর embeddings:

$$z_1, z_2, \ldots, z_T$$

Proposed quality-weighted track embedding:

$$z_{\text{track}} = \text{normalize}\left(\frac{\sum_{t=1}^{T} q_t z_t}{\sum_{t=1}^{T} q_t}\right)$$

Compare করবে:

```
Single frame
First frame
Best-quality frame
Average of all frames
Top-5 frames
Quality-weighted aggregation
```

Expected contribution:

> Quality-weighted multi-frame aggregation single-frame decision-এর চেয়ে stable এবং accurate।

---

32. Gallery Search with FAISS

Normalized embeddings হলে inner product cosine similarity-এর equivalent হিসেবে ব্যবহার করা যায়।

Basic index:

Python Run

```python
import faiss
import numpy as np


class FaceGallery:
    def __init__(self, embedding_dim: int = 512) -> None:
        self.index = faiss.IndexFlatIP(embedding_dim)
        self.subject_ids: list[str] = []

    def add(
        self,
        subject_id: str,
        embedding: np.ndarray,
    ) -> None:
        vector = embedding.astype("float32").reshape(1, -1)
        faiss.normalize_L2(vector)

        self.index.add(vector)
        self.subject_ids.append(subject_id)

    def search(
        self,
        embedding: np.ndarray,
        top_k: int = 2,
    ) -> list[tuple[str, float]]:
        vector = embedding.astype("float32").reshape(1, -1)
        faiss.normalize_L2(vector)
```

```
        scores, indices = self.index.search(vector, top_k)

        results: list[tuple[str, float]] = []

        for score, index in zip(scores[0], indices[0]):
            if index < 0:
                continue

            results.append(
                (self.subject_ids[index], float(score))
            )

        return results
```

---

33. Open-Set Decision

শুধু top similarity threshold যথেষ্ট নয়।

Use:

$$s_1 \geq \tau_s$$

$$s_1 - s_2 \geq \tau_m$$

$$q_{\text{track}} \geq \tau_q$$

এখানে:

```
s1 = highest similarity
s2 = second-highest similarity
τs = match threshold
τm = ambiguity margin
τq = minimum quality
```

Decision:

Python Run

```
def open_set_decision(
    top1_id: str,
    top1_score: float,
    top2_score: float,
    quality: float,
    similarity_threshold: float,
    margin_threshold: float,
    quality_threshold: float,
) -> str:
```

```
    if quality < quality_threshold:
        return "INSUFFICIENT_QUALITY"

    if top1_score < similarity_threshold:
        return "UNKNOWN"

    if top1_score - top2_score < margin_threshold:
        return "AMBIGUOUS"

    return top1_id
```

Threshold কখনো test set দেখে ঠিক করবে না।

Threshold selection:

```
Development validation set only
```

তারপর final test-এ unchanged threshold ব্যবহার করবে।

---

34. Complete Experiment List

Experiment A — Model Comparison

```
A1: InsightFace ArcFace
A2: AdaFace
A3: FaceLiVTv2-S
```

Same dataset, same crops, same splits।

Experiment B — Domain Adapter

```
B1: AdaFace without adapter
B2: AdaFace + residual adapter
```

Experiment C — Augmentation

```
C1: No augmentation
C2: Basic augmentation
C3: CCTV augmentation
C4: Combined CCTV augmentation
```

Experiment D — Enrollment

```
D1: One clean image
D2: Five clean images
D3: One image + augmentations
D4: Enrollment video
D5: Quality-weighted augmented template
```

Experiment E — Temporal Aggregation

```
E1: Single frame
E2: Best frame
E3: Mean embedding
E4: Top-5 mean
E5: Quality-weighted aggregation
```

Experiment F — Open-Set Decision

```
F1: Similarity threshold only
F2: Similarity + top1-top2 margin
F3: Similarity + margin + quality gate
```

Experiment G — CCTV Conditions

Separate results:

```
Near / Medium / Far
Normal light / Low light
Clear / Blurred
Frontal / Side pose
Unoccluded / Occluded
Low / Medium / High resolution
```

Experiment H — Runtime

Measure:

```
Detection latency
Embedding latency
Adapter latency
Gallery search latency
End-to-end latency
FPS
RAM usage
GPU memory
Model size
Gallery insertion time
```

---

35. Evaluation Metrics

শুধু accuracy লিখবে না।

Verification

```
ROC-AUC
EER
TAR at FAR = 1%
TAR at FAR = 0.1%
```

Open-Set Identification

```
Rank-1 Identification Rate
Rank-5 Identification Rate
TPIR at FPIR = 1%
TPIR at FPIR = 0.1%
```

Unknown Detection

```
AUROC
False Known Rate
False Unknown Rate
Unknown Rejection Rate
```

Detection

```
Precision
Recall
Average Precision
Detection rate by face size
```

Reliability

```
Mean across 3 splits
Standard deviation
95% bootstrap confidence interval
```

---

36. Complete Four-Month Roadmap

Week 1 — Basic Face Detection

কাজ:

```
Install tools
Create repository
Run SCRFD/InsightFace
Detect faces from images
Save bounding boxes
```

Deliverable:

```
01_face_detection_demo.ipynb
```

Week 2 — Embedding and Dynamic Gallery

কাজ:

```
Extract embeddings
Calculate cosine similarity
Create 5-person gallery
Add a new identity without retraining
Test known and unknown faces
```

Deliverable:

```
Working dynamic enrollment baseline
```

Week 3 — Research Protocol and Ethics

কাজ:

```
Finalize title
Finalize research questions
Prepare consent form
Prepare collection protocol
Get supervisor/institution approval
Create metadata templates
```

Deliverable:

```
Approved data-collection protocol
```

Data collection approval-এর আগে শুরু করবে না।

Week 4 — Pilot Dataset

Participants:

```
5 people
```

Capture:

```
Enrollment session
CCTV Session 1
CCTV Session 2
Two cameras
```

তারপর পুরো pipeline test করবে।

Deliverable:

```
Pilot CampusFace-BD
```

Pilot-এর ভুল দেখে main collection protocol ঠিক করবে।

Week 5 — Main Data Collection Part 1

Target:

```
First 30 participants
```

কাজ:

```
Enrollment capture
CCTV Session 1
Metadata verification
Daily backup
```

Week 6 — Main Data Collection Part 2

Target:

```
Remaining 30 participants
CCTV Session 2 for all participants
```

Deliverable:

```
60-participant raw dataset
```

Week 7 — Processing and Annotation

কাজ:

```
Extract 5 FPS frames
Run SCRFD
Align faces
Generate tracks
Manually verify tracks
Annotate 1,000 frames in CVAT
```

Deliverable:

```
CampusFace-BD v1 processed dataset
```

Week 8 — Baseline Experiments

কাজ:

```
ArcFace baseline
AdaFace baseline
Public QMUL-SurvFace baseline
Local dataset baseline
Threshold calibration
```

Deliverable:

```
Baseline tables and curves
```

Week 9 — CCTV Augmentation

কাজ:

```
Implement four severity levels
Generate individual degradations
Generate combined degradations
Verify identity preservation
Run augmentation experiments
```

Deliverable:

```
Augmentation ablation results
```

Week 10 — Domain Adapter

কাজ:

```
Prepare anchor-positive-negative triplets
Train residual adapter
Early stopping
Compare before/after adapter
```

Deliverable:

```
Best adapter checkpoint
```

Week 11 — Dynamic Enrollment Research

কাজ:

```
One-shot enrollment
Multi-shot enrollment
Augmented enrollment
Quality-weighted template
Gallery insertion benchmark
```

Deliverable:

```
Dynamic enrollment experiment
```

Week 12 — Tracking and Temporal Aggregation

কাজ:

```
Integrate ByteTrack
Create track embeddings
Implement top-K frames
Implement quality weighting
Compare aggregation methods
```

Deliverable:

```
Multi-frame recognition results
```

Week 13 — FaceLiVTv2 Comparison

কাজ:

```
Load pretrained FaceLiVTv2-S
Extract embeddings
Run same dataset splits
Compare accuracy, latency and model size
```

Deliverable:

```
CNN vs hybrid Transformer table
```

Week 14 — Final Experiments

কাজ:

```
Run all three identity splits
Generate confidence intervals
Run condition-wise evaluation
Run open-set evaluation
Run runtime benchmarks
```

Deliverable:

```
Final frozen results
```

এই সপ্তাহের পরে model বা threshold casually পরিবর্তন করবে না।

Week 15 — Paper Writing

Chapters:

```
Introduction
Related Work
Dataset and Protocol
Proposed Method
Experimental Setup
Results
Discussion
Ethics and Limitations
Conclusion
```

Figures:

```
Dataset collection diagram
Full architecture
Dynamic enrollment flow
Domain adapter architecture
Quality aggregation flow
ROC curve
Open-set curve
Condition-wise performance
```

Week 16 — Reproducibility and Demo

কাজ:

```
Run project from clean environment
Verify scripts
Freeze requirements
Prepare README
Prepare dataset card
Build minimal Streamlit demo
Proofread paper
Prepare presentation
```

37. প্রথম ৭ দিনের Exact Checklist

Day 1

```
Install Git
Install VS Code
Install Miniconda
Create private GitHub repository
Create folder structure
Create .gitignore
```

Commit:

```bash
git add .
git commit -m "Initialize DynaCampus-FR research repository"
git push
```

Day 2

```
Create Python environment
Install packages
Run environment check
Verify OpenCV
Verify PyTorch
Verify ONNX Runtime
```

Output file:

```
notebooks/00_environment_check.ipynb
```

Day 3

```
Run InsightFace/SCRFD
Test single-person image
Test group image
Save face boxes
Inspect landmarks
```

Output:

```
notebooks/01_face_detection_demo.ipynb
```

Day 4

```
Extract face embeddings
Compare same-person images
Compare different-person images
Save similarity scores in CSV
```

Output:

```
notebooks/03_embedding_similarity_demo.ipynb
```

Day 5

নিজেদের team-এর পাঁচজনের:

```
3 images per person
```

ব্যবহার করে gallery বানাবে।

Test:

```
Known image
Unknown image
Different pose
Low-light image
```

Output:

```
notebooks/04_dynamic_enrollment_demo.ipynb
```

Day 6

নতুন sixth person যোগ করবে:

```
No model retraining
Only new embedding template
```

Record করবে:

```
Enrollment processing time
Gallery insertion time
Recognition result
```

Day 7

একটি 20-second video-তে:

```
Detect faces frame-by-frame
Save embeddings
Average embeddings
Compare single-frame vs average
```

End-of-week demonstration:

> “The backbone remains fixed. New identities can be enrolled by inserting templates without retraining.”

---

38. পাঁচজন Team Member-এর Task Allocation

Member 1 — Dataset and Ethics Lead

```
Consent
Participant scheduling
Metadata
Backup
Dataset documentation
```

Member 2 — Detection and Tracking Lead

```
SCRFD
Alignment
Frame extraction
ByteTrack
CVAT annotation
```

Member 3 — Recognition Lead

```
ArcFace
AdaFace
```

```
FaceLiVTv2
Embedding extraction
Gallery
```

Member 4 — Research Method Lead

```
Domain Adapter
Augmentation
Quality scoring
Temporal aggregation
```

Member 5 — Evaluation and Paper Lead

```
Dataset splits
Metrics
Threshold calibration
Plots
Tables
Overleaf
```

সব member code review এবং data verification-এ অংশ নেবে।

---

39. Experiment Log Template

প্রতিটি experiment-এর জন্য একটি row:

csv

```csv
experiment_id,date,split,backbone,adapter,augmentation,enrollment,aggregation,threshold_v
EXP001,2026-08-10,split01,AdaFace,no,none,one_shot,single_frame,T01,42,baseline
```

Result file:

csv

```csv
experiment_id,rank1,tpir_fp01,unknown_auroc,eer,fps,latency_ms
EXP001,0.71,0.54,0.82,0.13,18.4,54.3
```

সব run-এর জন্য random seed:

Python Run

```python
import random
import numpy as np
import torch
```

```python
def set_seed(seed: int = 42) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

---

40. Paper Architecture

Chapter 1 — Introduction

```
Problem
Motivation
Research gap
Research questions
Contributions
```

Chapter 2 — Related Work

```
Surveillance face recognition
Open-set recognition
Dynamic enrollment
Low-quality face recognition
Quality-aware recognition
Video-based aggregation
CNN vs Vision Transformer
```

Chapter 3 — CampusFace-BD

```
Participants
Ethics
Camera setup
Sessions
Conditions
Metadata
Splits
```

Chapter 4 — Proposed Method

```
SCRFD detection
AdaFace backbone
```

```
Residual Domain Adapter
Augmented enrollment
Quality calculation
Temporal aggregation
Open-set decision
```

Chapter 5 — Experiments

```
Datasets
Models
Training settings
Metrics
Baselines
Ablations
Runtime
```

Chapter 6 — Results and Discussion

```
Main results
Condition-wise results
Failure analysis
Transformer comparison
Limitations
```

Chapter 7 — Conclusion

---

41. তোমাদের Final Motivation-এর Main Logic

তোমাদের motivation attendance app নয়।

Motivation হবে:

> Universities have continuously changing populations, while existing classification-based face-recognition prototypes assume a fixed set of identities and require retraining when new users are introduced. At the same time, clean enrollment photographs and low-quality CCTV probes belong to substantially different visual domains. Therefore, a campus recognition system requires training-free identity enrollment, domain adaptation, quality-aware video processing and calibrated open-set rejection rather than a fixed closed-set classifier.

---

42. কোন কাজগুলো একদম করবে না

```
Seven-class softmax CNN দিয়ে final result দেবে না
LFW-কে main CCTV dataset বলবে না
40×40 input ব্যবহার করবে না
Sobel/Canny image recognizer-এর input হিসেবে দেবে না
Test data দেখে threshold নির্বাচন করবে না
Adjacent frames random train/test split করবে না
Unknown people-কে different-looking dataset দিয়েই শুধু test করবে না
Face data GitHub-এ upload করবে না
Consent ছাড়া campus footage collect করবে না
Automatic uncertain prediction দিয়ে gallery update করবে না
Desktop/mobile app-এ সময় নষ্ট করবে না
RAG core pipeline-এ যোগ করবে না
FaceLiVTv2-কে peer-reviewed final model বলে দাবি করবে না
শুধু accuracy report করবে না
```

---

43. তোমার এখনই প্রথম কাজ

আজ শুধু নিচের চারটি কাজ শেষ করো:

```
1. Private GitHub repository তৈরি
2. Folder structure তৈরি
3. Python environment setup
4. InsightFace দিয়ে একটি image-এ face detect
```

Data collection, AdaFace training, ViT বা Domain Adapter দিয়ে শুরু করবে না।

তোমার first milestone হবে:

```
Input image
    ↓
Face detected
    ↓
Face aligned
    ↓
512-D embedding extracted
    ↓
Same-person and different-person similarity compared
```

এই foundation working না হওয়া পর্যন্ত পরের stage-এ যাবে না।